scikit-learn

Lecture 12

Dr. Colin Rundel

scikit-learn

> Scikit-learn is an open source machine learning library that supports supervised and unsupervised learning. It also provides various tools for model fitting, data preprocessing, model selection, model evaluation, and many other utilities.
>
> - Simple and efficient tools for predictive data analysis
> - Accessible to everybody, and reusable in various contexts
> - Built on NumPy, SciPy, and matplotlib
> - Open source, commercially usable - BSD license

This is one of several other “scikits” (e.g. scikit-image) which are scientific toolboxes built on top of scipy. For a more

Installation

To add scikit-learn to your python environment, you can use:

```
> uv add scikit-learn
```

Previously you could also use `sklearn` as the package name, but this is no longer supported and will result in an error.

Submodules

The `sklearn` package contains a large number of submodules which are specialized for different tasks / models,

- `sklearn.base` - Base classes and utility functions
- `sklearn.calibration` - Probability Calibration
- `sklearn.cluster` - Clustering
- `sklearn.compose` - Composite Estimators
- `sklearn.covariance` - Covariance Estimators
- `sklearn.cross_decomposition` - Cross decomposition
- `sklearn.datasets` - Datasets
- `sklearn.decomposition` - Matrix Decomposition
- `sklearn.discriminant_analysis` - Discriminant Analysis
- `sklearn.ensemble` - Ensemble Methods
- `sklearn.exceptions` - Exceptions and warnings
- `sklearn.experimental` - Experimental
- `sklearn.feature_extraction` - Feature Extraction
- `sklearn.feature_selection` - Feature Selection
- `sklearn.gaussian_process` - Gaussian Processes
- `sklearn.impute` - Impute
- `sklearn.inspection` - Inspection
- `sklearn.isotonic` - Isotonic regression
- `sklearn.kernel_approximation` - Kernel Approximation
- `sklearn.kernel_ridge` - Kernel Ridge Regression
- `sklearn.linear_model` - Linear Models
- `sklearn.manifold` - Manifold Learning
- `sklearn.metrics` - Metrics
- `sklearn.mixture` - Gaussian Mixture Models
- `sklearn.model_selection` - Model Selection
- `sklearn.multiclass` - Multiclass classification
- `sklearn.multioutput` - Multioutput regression and classification
- `sklearn.naive_bayes` - Naive Bayes
- `sklearn.neighbors` - Nearest Neighbors
- `sklearn.neural_network` - Neural network models
- `sklearn.pipeline` - Pipeline
- `sklearn.preprocessing` - Preprocessing and Normalization
- `sklearn.random_projection` - Random projection
- `sklearn.semi_supervised` - Semi-Supervised Learning
- `sklearn.svm` - Support Vector Machines
- `sklearn.tree` - Decision Trees
- `sklearn.utils` - Utilities

Model Fitting

Sample data

To begin, we will examine a simple data set on the size and weight of a number of books. The goal is to model the weight of a book using some combination of the other features in the data.

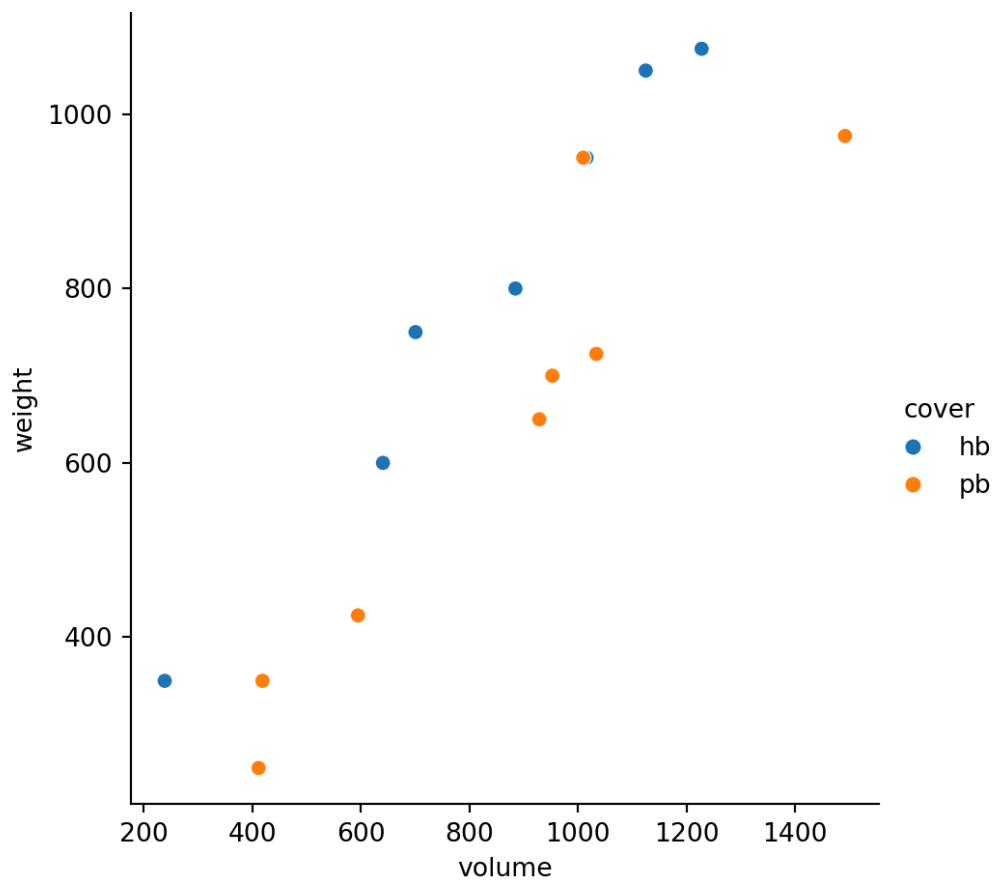
The included columns are:

- `volume` - book volumes in cubic centimeters
- `weight` - book weights in grams
- `cover` - a categorical variable with levels `"hb"` hardback, `"pb"` paperback

```
books = pd.read_csv("data/daag_books.csv");
```

	volume	weight	cover
0	885	800	hb
1	1016	950	hb
2	1125	1050	hb
3	239	350	hb
4	701	750	hb
5	641	600	hb
6	1228	1075	hb
7	412	250	pb
8	953	700	pb
9	929	650	pb
10	1492	975	pb
11	419	350	pb
12	1010	950	pb
13	595	425	pb
14	1034	725	pb

```python
g = sns.relplot(data=books, x="volume", y="weight", hue="cover")
```



Linear regression

scikit-learn uses an object oriented system for implementing the various modeling approaches, the class for `LinearRegression` is part of the `linear_model` submodule.

```
from sklearn.linear_model import LinearRegression
```

Each modeling class needs to be constructed (potentially with options) and then the resulting object will provide attributes and methods.

```
lm = LinearRegression()

m = lm.fit(
  X = books[["volume"]],
  y = books.weight
)
```

```
m.coef_
```

```
array([0.70863714])
```

```
m.intercept_
```

```
np.float64(107.67931061376612)
```

Note `lm` and `m` are labels for the same underlying `LinearRegression` object,

```
lm.coef_
```

```
array([0.70863714])
```

```
lm.intercept_
```

```
np.float64(107.67931061376612)
```

A couple of considerations

When fitting a model, scikit-learn expects `X` to be a 2d array-like object (e.g. a `np.array` or `pd.DataFrame`), so it will not accept objects like a `pd.Series` or 1d `np.array`.

```
lm.fit(
  X = books.volume,
  y = books.weight
)
```

```
ValueError: Expected a 2-dimensional container b
```

```
lm.fit(
  X = np.array(books.volume),
  y = books.weight
)
```

```
ValueError: Expected 2D array, got 1D array inst
array=[ 885 1016 1125  239  701  641 1228  412
 1034].
Reshape your data either using array.reshape(-1,
```

```
lm.fit(
  X = np.array(books.volume).reshape(-1,1),
  y = books.weight
)
```

Model parameters

Depending on the model being used, there will be a number of parameters that can be configured when constructing the model object or via the `set_params()` method.

```
lm.get_params()
```

```
{'copy_X': True, 'fit_intercept': True, 'n_jobs': None, 'positive': False, 'tol': 1e-0
```

```
lm.set_params(fit_intercept = False)
```

```
▾ LinearRegression
▸ Parameters
```

```
lm = lm.fit(X = books[["volume"]], y = books.weight)
lm.intercept_
```

```
0.0
```

```
lm.coef_
```

```
array([0.81932487])
```

Model prediction

Once the model coefficients have been fit, it is possible to predict from the model via the `predict()` method, this method requires a matrix-like `X` as input and in the case of `LinearRegression` returns an array of predicted y values.

```
lm.predict(X = books[["volume"]])
```

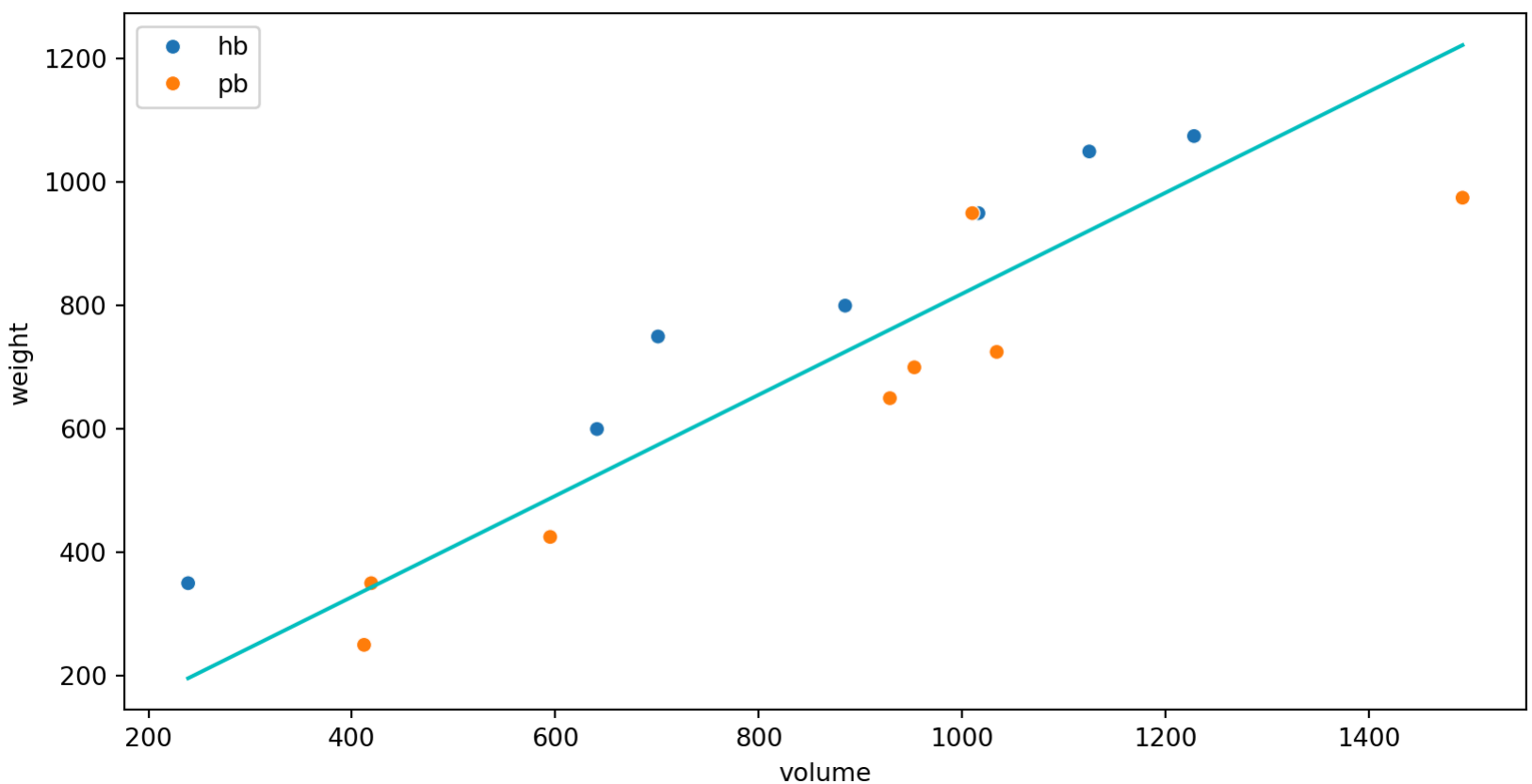
```
array([ 725.10251417,  832.43407276,  921.74048411,  195.81864507,
        574.34673721,  525.18724472, 1006.13094621,  337.5618484 ,
        780.81660565,  761.15280865, 1222.43271315,  343.29712253,
        827.51812351,  487.49830048,  847.1819205 ])
```

```
books = books.assign(
  weight_lm_pred = lambda x: lm.predict(X = x[["volume"]])
)
books
```

	volume	weight	cover	weight_lm_pred
0	885	800	hb	725.102514
1	1016	950	hb	832.434073
2	1125	1050	hb	921.740484
3	239	350	hb	195.818645
4	701	750	hb	574.346737
5	641	600	hb	525.187245
6	1228	1075	hb	1006.130946

```
7      412     250    pb      337.561848
8      953     700    pb      780.816606
9      929     650    pb      761.152809
10    1492     975    pb     1222.432713
11     419     350    pb      343.297123
12    1010     950    pb      827.518124
13     595     425    pb      487.498300
14    1034     725    pb      847.181921
```

```python
plt.figure()
sns.scatterplot(data=books, x="volume", y="weight", hue="cover")
sns.lineplot(data=books, x="volume", y="weight_lm_pred", color="c")
plt.show()
```

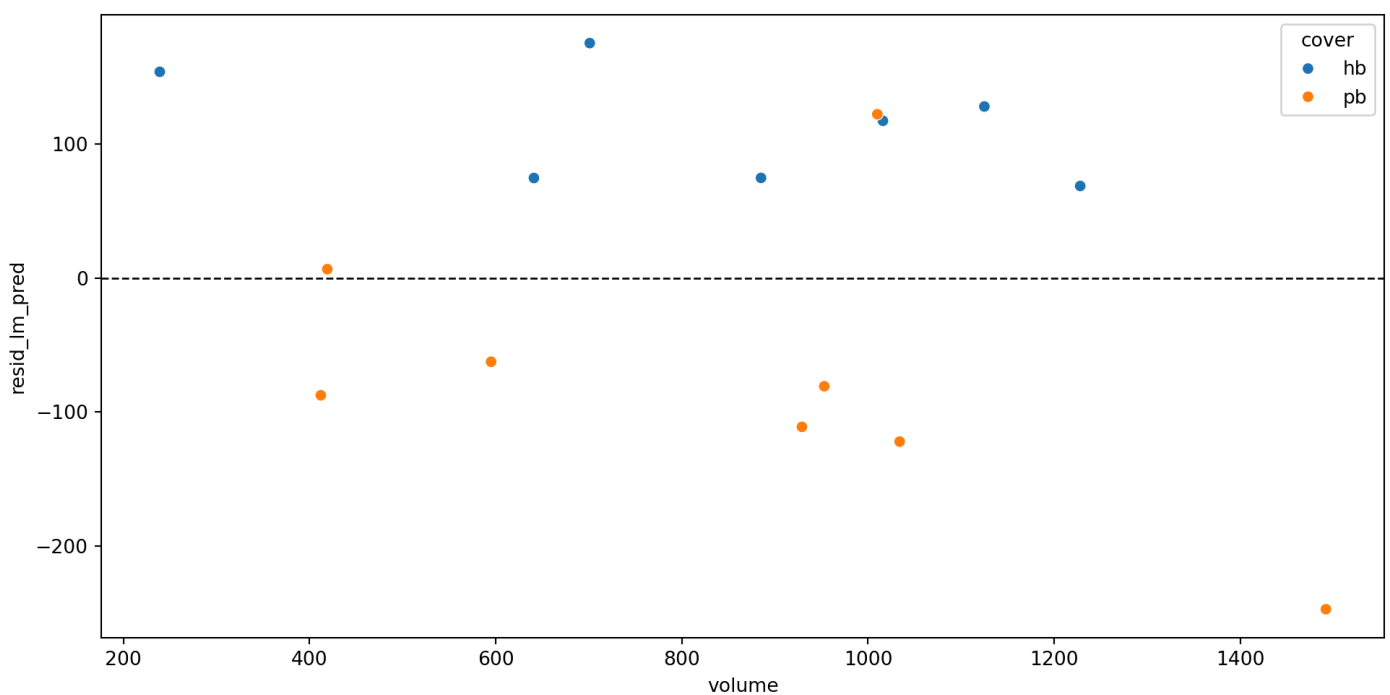


Residuals?

There is no built in functionality for calculating residuals, so this needs to be done by hand.

```
books["resid_lm_pred"] = books["weight"] - books["weight_lm_pred"]
```

```
plt.figure(layout="constrained")
ax = sns.scatterplot(data=books, x="volume", y="resid_lm_pred", hue="cover")
ax.axhline(c="k", ls="--", lw=1)
plt.show()
```



Categorical variables?

Scikit-learn expects that the model matrix be numeric before fitting,

```
lm = lm.fit(
  X = books[["volume", "cover"]],
  y = books.weight
)
```

```
ValueError: could not convert string to float: 'hb'
```

the solution here is to dummy code the categorical variables - this can be done with pandas via `pd.get_dummies()` or with a scikit-learn preprocessor.

```
pd.get_dummies(books[["volume", "cover"]])
```

```
    volume  cover_hb  cover_pb
0      885      True     False
1     1016      True     False
2     1125      True     False
3      239      True     False
4      701      True     False
5      641      True     False
6     1228      True     False
7      412     False      True
8      953     False      True
9      929     False      True
```

```
10    1492     False      True
11     419     False      True
12    1010     False      True
13     595     False      True
14    1034     False      True
```

Dummy coded model

```
lm = LinearRegression().fit(
  X = pd.get_dummies(books[["volume", "cover"]]),
  y = books.weight
)
```

```
lm.intercept_
```

```
np.float64(105.93920788192202)
```

```
lm.coef_
```

```
array([  0.71795374,  92.02363569, -92.02363569])
```

Do the above results look reasonable? What went wrong?

Quick comparison with R

```r
d = read.csv('data/daag_books.csv')
d['cover_hb'] = ifelse(d$cover == "hb", 1, 0)
d['cover_pb'] = ifelse(d$cover == "pb", 1, 0)
lm = lm(weight~volume+cover_hb+cover_pb, data=d)
summary(lm)
```

```
Call:
lm(formula = weight ~ volume + cover_hb + cover_pb, data = d)

Residuals:
    Min      1Q  Median      3Q     Max 
-110.10  -32.32  -16.10   28.93  210.95 

Coefficients: (1 not defined because of singularities)
             Estimate Std. Error t value Pr(>|t|)    
(Intercept)  13.91557   59.45408   0.234 0.818887    
volume        0.71795    0.06153  11.669  6.6e-08 ***
cover_hb    184.04727   40.49420   4.545 0.000672 ***
cover_pb           NA         NA      NA       NA    
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 78.2 on 12 degrees of freedom
Multiple R-squared:  0.9275,	Adjusted R-squared:  0.9154 
F-statistic: 76.73 on 2 and 12 DF,  p-value: 1.455e-07
```

Avoiding co-linearity

```
lm1 = LinearRegression(
  fit_intercept = False
).fit(
  X = pd.get_dummies(
    books[["volume", "cover"]]
  ),
  y = books.weight
)
```

```
lm1.intercept_
```

```
0.0
```

```
lm1.coef_
```

```
array([  0.71795374, 197.96284357,  13.915
```

```
lm1.feature_names_in_
```

```
array(['volume', 'cover_hb', 'cover_pb'],
```

```
lm2 = LinearRegression(
  fit_intercept = True
).fit(
  X = pd.get_dummies(
    books[["volume", "cover"]],
    drop_first=True
  ),
  y = books.weight
)
```

```
lm2.intercept_
```

```
np.float64(197.96284357271747)
```

```
lm2.coef_
```

```
array([   0.71795374, -184.04727138])
```

```
lm2.feature_names_in_
```

```
array(['volume', 'cover_pb'], dtype=object
```

Preprocessors

Preprocessors

These are a collection of transformer classes present in the `sklearn.preprocessing` submodule that are designed to help with the preparation of raw feature data into quantities more suitable for downstream modeling tools.

Like the modeling classes, they have an object oriented design that shares a common interface (methods and attributes) for bringing in data, transforming it, and returning it.

OneHotEncoder

For dummy coding we can use the `OneHotEncoder` preprocessor, the default is to use one hot encoding but standard dummy coding can be achieved via the `drop` parameter.

```
from sklearn.preprocessing import OneHotEncoder
```

```python
enc = OneHotEncoder(sparse_output=False)
enc.fit(X = books[["cover"]])
```

▾ OneHotEncoder ⓘ ?

▸ Parameters

```python
enc.transform(X = books[["cover"]])
```

```
array([[1., 0.],
       [1., 0.],
       [1., 0.],
       [1., 0.],
       [1., 0.],
       [1., 0.],
       [1., 0.],
       [0., 1.],
       [0., 1.],
       [0., 1.],
       [0., 1.],
       [0., 1.],
       [0., 1.],
       [0., 1.],
       [0., 1.]])
```

```python
enc = OneHotEncoder(sparse_output=False, dro
enc.fit_transform(X = books[["cover"]])
```

```
array([[0.],
       [0.],
       [0.],
       [0.],
       [0.],
       [0.],
       [0.],
       [1.],
       [1.],
       [1.],
       [1.],
       [1.],
       [1.],
       [1.],
       [1.]])
```

Other useful bits

```python
enc.get_feature_names_out()
```

```
array(['cover_hb', 'cover_pb'], dtype=object)
```

```python
f = enc.transform(X = books[["cover"]])
f
```

```
array([[1., 0.],
       [1., 0.],
       [1., 0.],
       [1., 0.],
       [1., 0.],
       [1., 0.],
       [1., 0.],
       [0., 1.],
       [0., 1.],
       [0., 1.],
       [0., 1.],
       [0., 1.],
       [0., 1.],
       [0., 1.],
       [0., 1.],
       [0., 1.]])
```

```python
enc.inverse_transform(f)
```

```
array([['hb'],
       ['hb'],
       ['hb'],
       ['hb'],
       ['hb'],
       ['hb'],
       ['hb'],
       ['pb'],
       ['pb'],
       ['pb'],
       ['pb'],
       ['pb'],
       ['pb'],
       ['pb'],
       ['pb'],
       ['pb']], dtype=object)
```

A cautionary note

Unlike `pd.get_dummies()` it is not safe to use `OneHotEncoder` with both numerical and categorical features, as the former will also be transformed.

```
enc = OneHotEncoder(sparse_output=False)
X = enc.fit_transform(X = books[["volume", "cover"]])
pd.DataFrame(data=X, columns = enc.get_feature_names_out())
```

```
    volume_239  volume_412  volume_419  ...  volume_1492  cover_hb  cover_pb
0          0.0         0.0         0.0  ...          0.0       1.0       0.0
1          0.0         0.0         0.0  ...          0.0       1.0       0.0
2          0.0         0.0         0.0  ...          0.0       1.0       0.0
3          1.0         0.0         0.0  ...          0.0       1.0       0.0
4          0.0         0.0         0.0  ...          0.0       1.0       0.0
5          0.0         0.0         0.0  ...          0.0       1.0       0.0
6          0.0         0.0         0.0  ...          0.0       1.0       0.0
7          0.0         1.0         0.0  ...          0.0       0.0       1.0
8          0.0         0.0         0.0  ...          0.0       0.0       1.0
9          0.0         0.0         0.0  ...          0.0       0.0       1.0
10         0.0         0.0         0.0  ...          1.0       0.0       1.0
11         0.0         0.0         1.0  ...          0.0       0.0       1.0
12         0.0         0.0         0.0  ...          0.0       0.0       1.0
13         0.0         0.0         0.0  ...          0.0       0.0       1.0
14         0.0         0.0         0.0  ...          0.0       0.0       1.0

[15 rows x 17 columns]
```

Putting it together

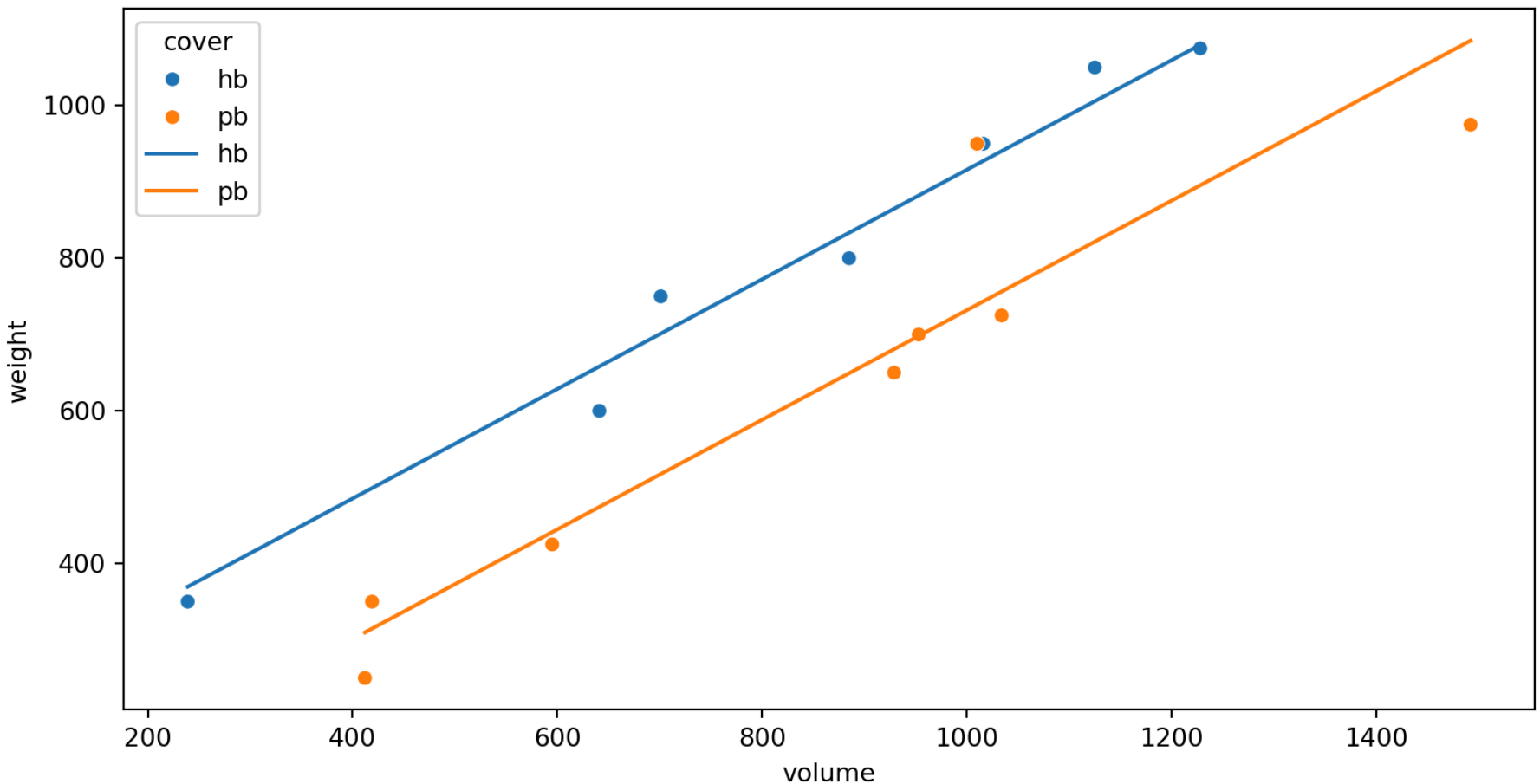
```python
cover = OneHotEncoder(
  sparse_output=False
).fit_transform(
  books[["cover"]]
)
X = np.c_[books.volume, cover]

lm2 = LinearRegression(
  fit_intercept=False
).fit(
  X = X,
  y = books.weight
)

lm2.coef_
```

```
array([  0.71795374, 197.96284357,  13.91557219]
```

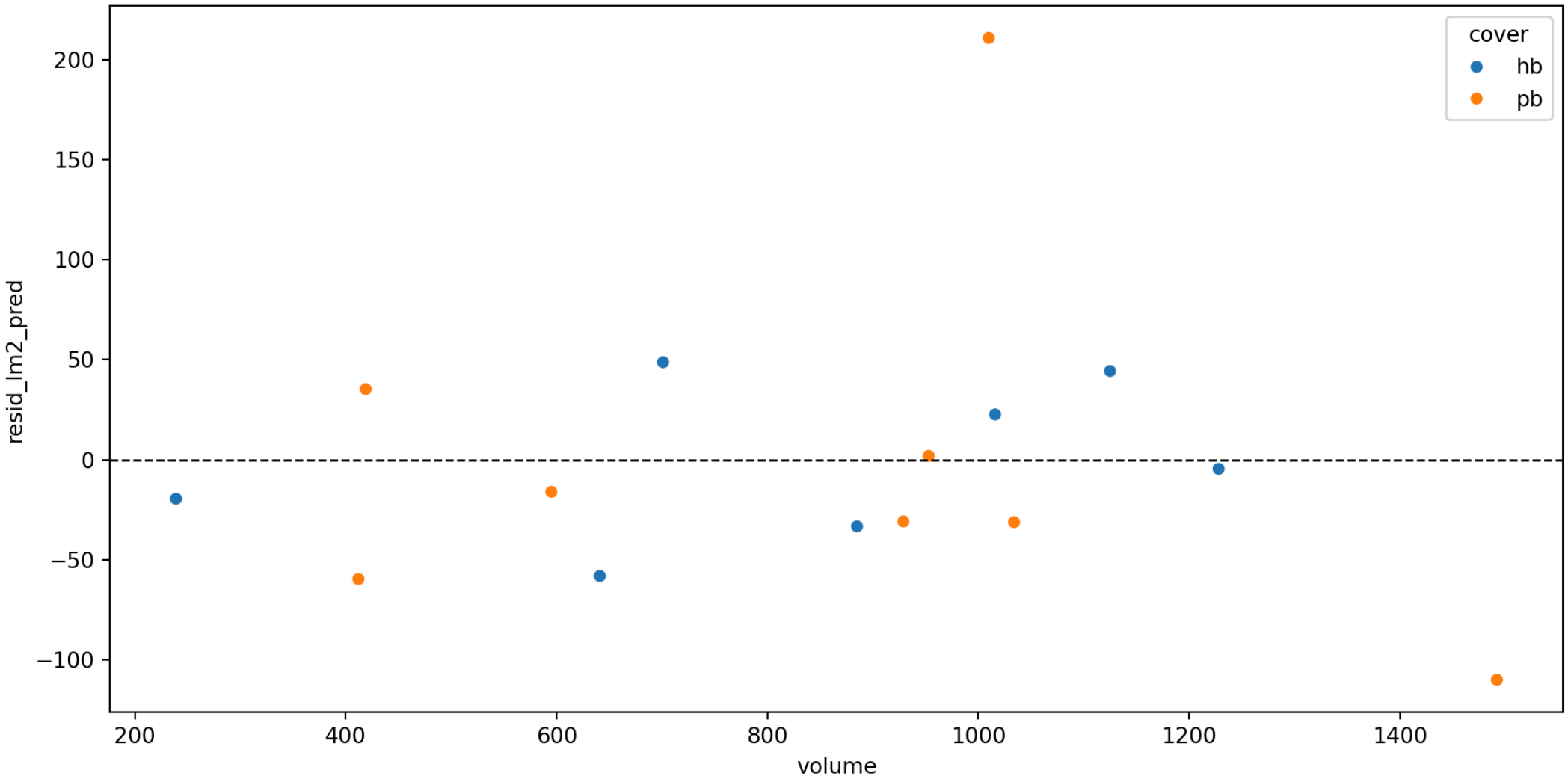
```python
books["weight_lm2_pred"] = lm2.predict(X=X)
books.drop(
  ["weight_lm_pred", "resid_lm_pred"],
  axis=1
)
```

	volume	weight	cover	weight_lm2_pred
0	885	800	hb	833.351907
1	1016	950	hb	927.403847
2	1125	1050	hb	1005.660805
3	239	350	hb	369.553788
4	701	750	hb	701.248418
5	641	600	hb	658.171193
6	1228	1075	hb	1079.610041
7	412	250	pb	309.712515
8	953	700	pb	698.125490
9	929	650	pb	680.894600
10	1492	975	pb	1085.102558
11	419	350	pb	314.738191
12	1010	950	pb	739.048853
13	595	425	pb	441.098050
14	1034	725	pb	756.279743

Model fit


cover
hb
pb
hb
pb
1000
800
600
400
weight
200
400
600
800
1000
1200
1400
volume

Model residuals



Model performance

Scikit-learn comes with a number of builtin functions for measuring model performance in the `sklearn.metrics` submodule - these are generally just functions that take the vectors `y_true` and `y_pred` and return a scalar score.

```
import sklearn.metrics as metrics
```

```
metrics.r2_score(books.weight, books.weight_
```

```
0.7800969547785039
```

```
metrics.r2_score(books.weight, books.weight_
```

```
0.927477575682168
```

```
metrics.mean_squared_error(
  books.weight, books.weight_lm_pred
)
```

```
14833.682083774476
```

```
metrics.mean_squared_error(
  books.weight, books.weight_lm2_pred
)
```

```
4892.04042259509
```

```
metrics.root_mean_squared_error(
  books.weight, books.weight_lm_pred
)
```

```
121.79360444528471
```

```
metrics.root_mean_squared_error(
  books.weight, books.weight_lm2_pred
)
```

```
69.94312276839725
```

Exercise 1

Create and fit a model for the `books` data that includes an interaction effect between `volume` and `cover`.

You will need to do this manually with `pd.get_dummies()` and some additional data munging.

The data can be read into pandas with,

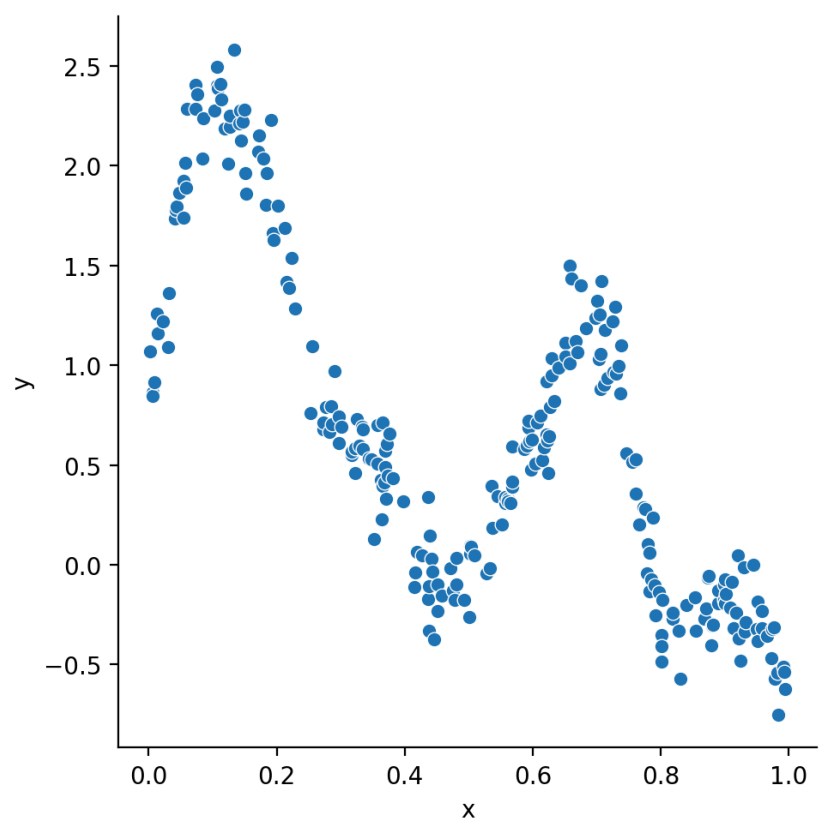
```
books = pd.read_csv(
  "https://sta663-sp26.github.io/slides/data/daag_books.csv"
)
```

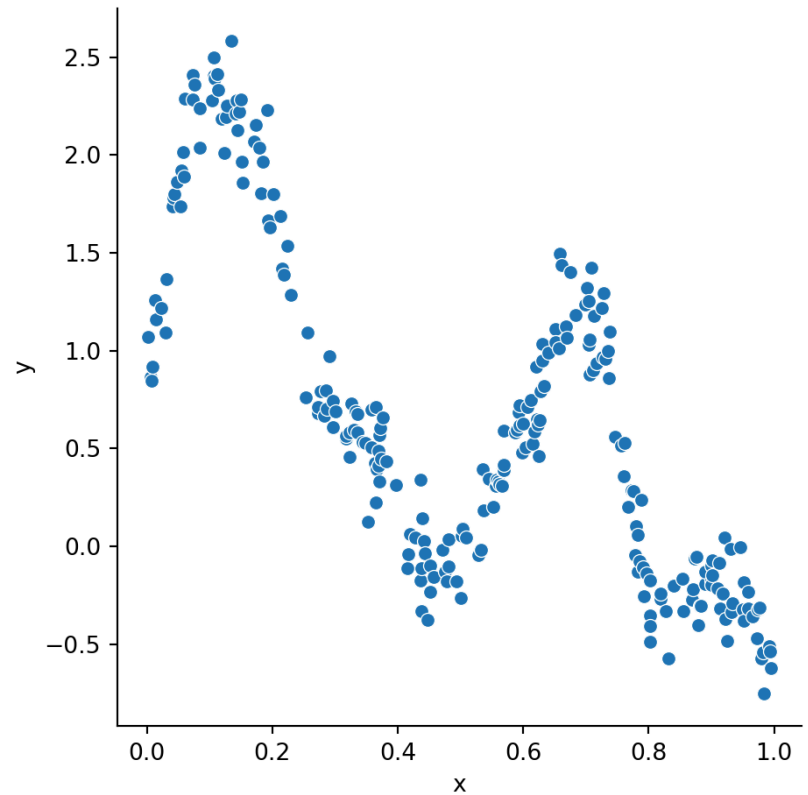
Other transformers

Polynomial regression

We will now look at another flavor of regression model, that involves preprocessing and a hyperparameter - namely polynomial regression.

```
df = pd.read_csv("data/gp.csv")
sns.relplot(data=df, x="x", y="y")
```




2.5
2.0
1.5
1.0
0.5
0.0
−0.5
y
0.0
0.2
0.4
0.6
0.8
1.0
x

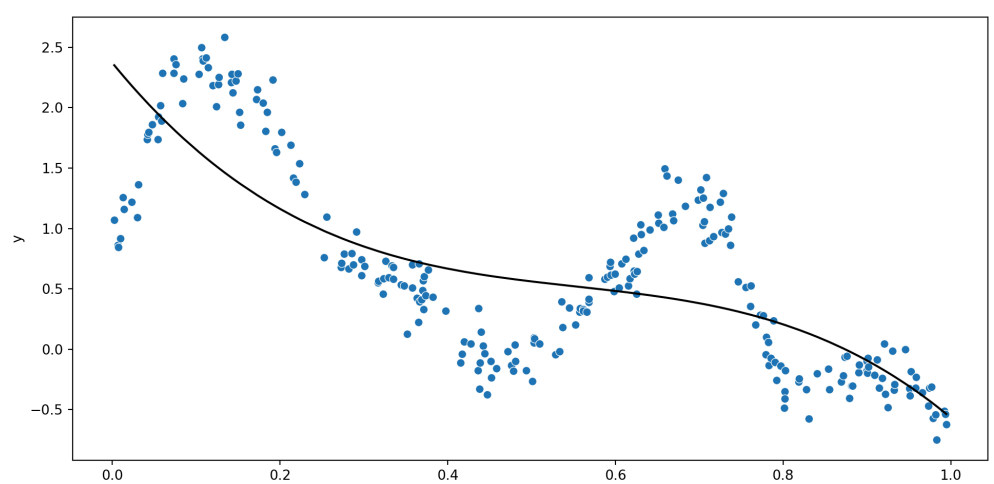
By hand

It is certainly possible to construct the necessary model matrix by hand (or even use a function to automate the process), but this is less than desirable generally - particularly if we want to do anything fancy (e.g. cross validation)

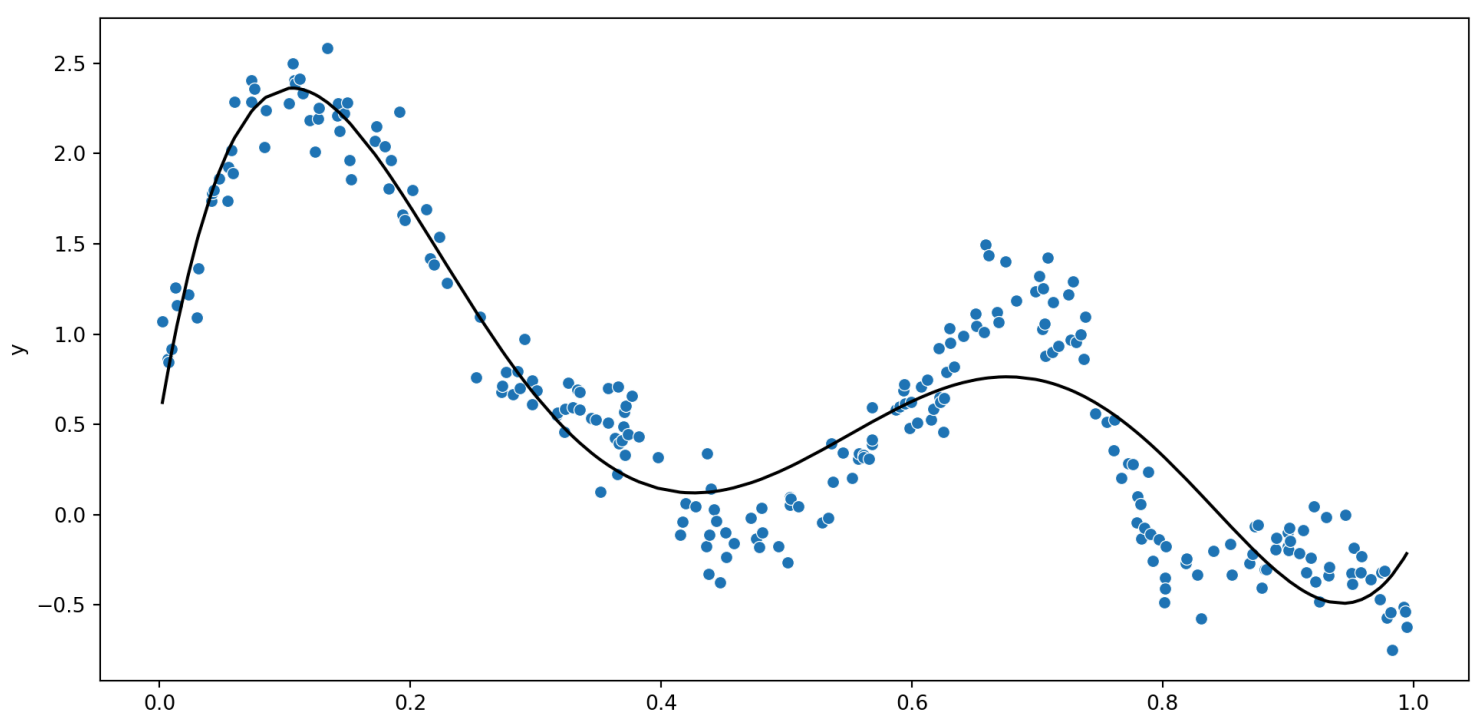
```python
X = np.c_[
    np.ones(df.shape[0]),
    df.x,
    df.x**2,
    df.x**3
]

plm = LinearRegression(
  fit_intercept = False
).fit(
  X=X, y=df.y
)

plm.coef_
```

```
array([ 2.36985684, -8.49429068, 13.950663
```

```python
df["y_pred"] = plm.predict(X=X)

plt.figure(layout="constrained")
sns.scatterplot(data=df, x="x", y="y")
sns.lineplot(data=df, x="x", y="y_pred
plt.show()
```



```python
X = np.c_[
    np.ones(df.shape[0]), df.x,
    df.x**2, df.x**3,
    df.x**4, df.x**5
]

plm = LinearRegression(
  fit_intercept = False
).fit(
  X=X, y=df.y
)
df["y_pred"] = plm.predict(X=X)
```



PolynomialFeatures

This is another transformer class from `sklearn.preprocessing` that simplifies the process of constructing polynomial features for your model matrix. Usage is similar to that of `OneHotEncoder`.

```
from sklearn.preprocessing import PolynomialFeatures
X = np.array(range(6)).reshape(-1,1)
```

```
pf = PolynomialFeatures(degree=3)
pf = pf.fit(X)
pf.transform(X)
```

```
array([[  1.,   0.,   0.,   0.],
       [  1.,   1.,   1.,   1.],
       [  1.,   2.,   4.,   8.],
       [  1.,   3.,   9.,  27.],
       [  1.,   4.,  16.,  64.],
       [  1.,   5.,  25., 125.]])
```

```
pf.get_feature_names_out()
```

```
array(['1', 'x0', 'x0^2', 'x0^3'], dtype=o
```

```
pf = PolynomialFeatures(
  degree=2, include_bias=False
)
pf.fit_transform(X)
```

```
array([[ 0.,  0.],
       [ 1.,  1.],
       [ 2.,  4.],
       [ 3.,  9.],
       [ 4., 16.],
       [ 5., 25.]])
```

```
pf.get_feature_names_out()
```

```
array(['x0', 'x0^2'], dtype=object)
```

Interactions

If the feature matrix `X` has more than one column then `PolynomialFeatures` transformer will include interaction terms with total degree up to `degree`.

```
X.reshape(-1, 2)
```

```
array([[0, 1],
       [2, 3],
       [4, 5]])
```

```
pf = PolynomialFeatures(
  degree=2, include_bias=False
)
pf.fit_transform(
  X.reshape(-1, 2)
)
```

```
array([[ 0.,  1.,  0.,  0.,  1.],
       [ 2.,  3.,  4.,  6.,  9.],
       [ 4.,  5., 16., 20., 25.]])
```

```
pf.get_feature_names_out()
```

```
array(['x0', 'x1', 'x0^2', 'x0 x1', 'x1^2'], dty
```

```
X.reshape(-1, 3)
```

```
array([[0, 1, 2],
       [3, 4, 5]])
```

```
pf = PolynomialFeatures(
  degree=2, include_bias=False
)
pf.fit_transform(
  X.reshape(-1, 3)
)
```

```
array([[ 0.,  1.,  2.,  0.,  0.,  0.,  1.,  2.,
       [ 3.,  4.,  5.,  9., 12., 15., 16., 20.,
```

```
pf.get_feature_names_out()
```

```
array(['x0', 'x1', 'x2', 'x0^2', 'x0 x1', 'x0 x2
       'x2^2'], dtype=object)
```

Modeling with PolynomialFeatures

```
def poly_model(X, y, degree):
  X  = PolynomialFeatures(
    degree=degree, include_bias=False
  ).fit_transform(
    X=X
  )
  y_pred = LinearRegression(
  ).fit(
    X=X, y=y
  ).predict(
    X
  )
  return metrics.root_mean_squared_error(y,
```

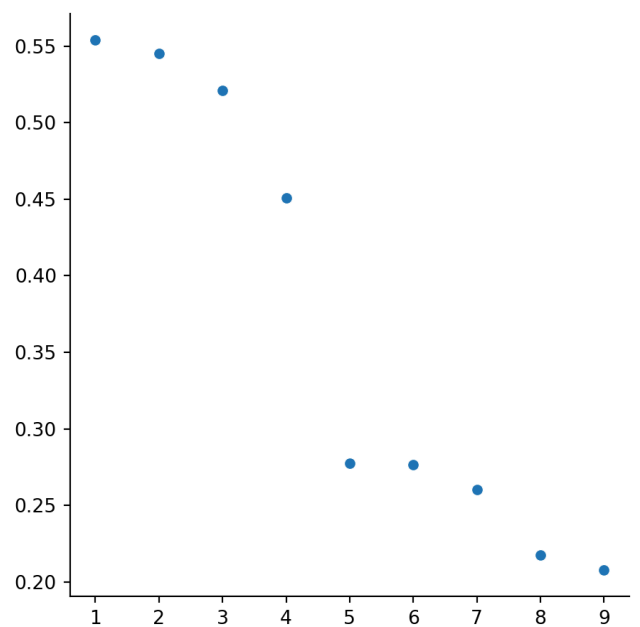
```
poly_model(X = df[["x"]], y = df.y, degree =
```

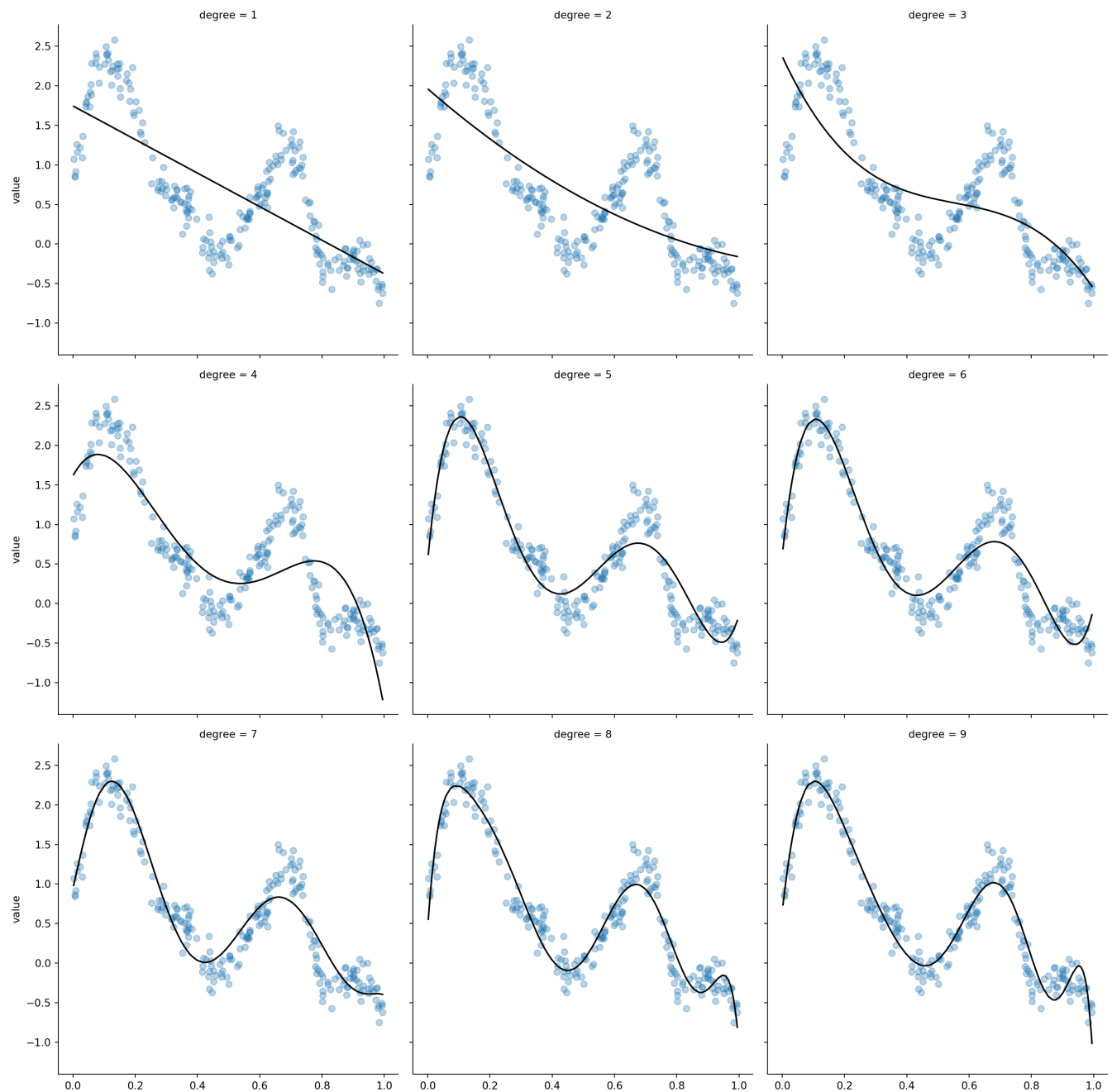
0.5449418707295371

```
poly_model(X = df[["x"]], y = df.y, degree =
```

0.5208157900621085

```
degrees = range(1,10)
rmses = [
  poly_model(X=df[["x"]], y=df.y, degree=d)
  for d in degrees
]
g = sns.relplot(x=degrees, y=rmses)
```




degree = 1
degree = 2
degree = 3
degree = 4
degree = 5
degree = 6
degree = 7
degree = 8
degree = 9
value
x
2.5
2.0
1.5
1.0
0.5
0.0
−0.5
−1.0
0.0
0.2
0.4
0.6
0.8
1.0

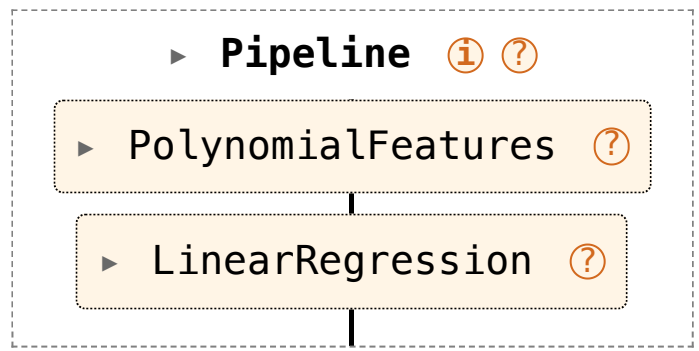
Pipelines

Pipelines

You may have noticed that `PolynomialFeatures` takes a model matrix as input and returns a new model matrix as output which is then used as the input for `LinearRegression`. This is not an accident, and by structuring the library in this way sklearn is designed to enable the connection of these steps together, into what sklearn calls a *pipeline*.

```python
from sklearn.pipeline import make_pipeline

p = make_pipeline(
  PolynomialFeatures(degree=4),
  LinearRegression()
)
```

```python
p
```



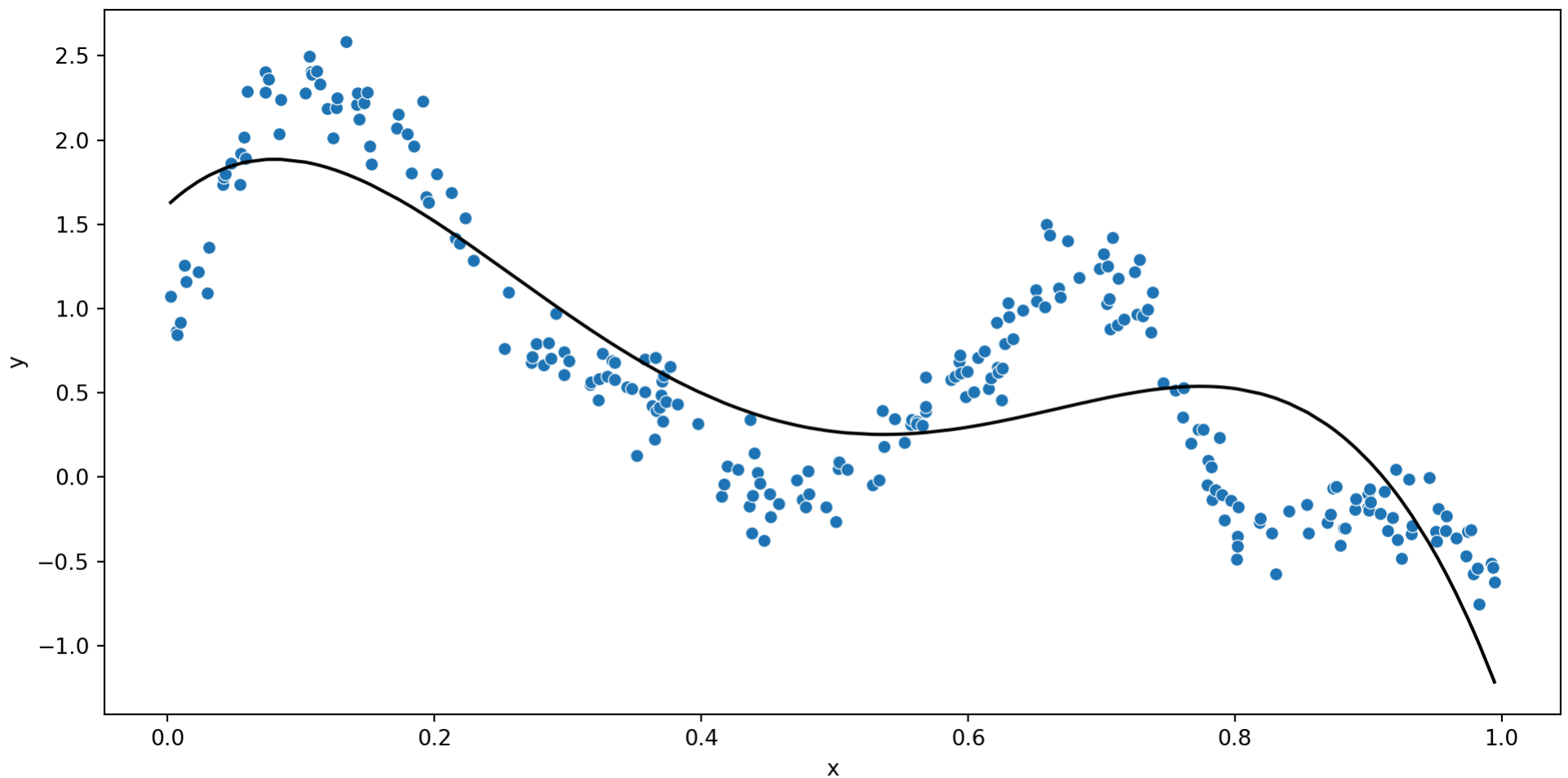
Using Pipelines

Once constructed, this object can be used just like our previous `LinearRegression` model (i.e. fit to our data and then used for prediction)

```
p = p.fit(X = df[["x"]], y = df.y)
p.predict(X = df[["x"]])
```

```
array([ 1.6295693 ,  1.65734929,  1.6610466 ,  1.67779767,  1.69667491,
        1.70475286,  1.75280126,  1.78471392,  1.79049912,  1.82690007,
        1.82966357,  1.83376043,  1.84494343,  1.86002819,  1.86228095,
        1.86619112,  1.86837909,  1.87065283,  1.88417882,  1.8844024 ,
        1.88527174,  1.88577463,  1.88544367,  1.86890805,  1.86365035,
        1.86252922,  1.86047349,  1.85377801,  1.84937708,  1.83754576,
        1.82623453,  1.82024199,  1.81799793,  1.79767794,  1.77255319,
        1.77034143,  1.76574288,  1.75371272,  1.74389585,  1.73804309,
        1.73356954,  1.65527727,  1.64812184,  1.61867613,  1.6041325 ,
        1.5960389 ,  1.56080881,  1.55036459,  1.54004364,  1.50903953,
        1.45096594,  1.43589836,  1.41886389,  1.39423307,  1.36180712,
        1.23072992,  1.21355164,  1.11776117,  1.11522002,  1.09595388,
        1.06449719,  1.04672121,  1.03662739,  1.01407206,  0.98208703,
        0.98081577,  0.96176797,  0.87491417,  0.87117573,  0.84223005,
        0.84171166,  0.82875003,  0.8085086 ,  0.79166069,  0.78167248,
        0.78078036,  0.73538157,  0.7181484 ,  0.70046945,  0.67233502,
        0.67229069,  0.64782899,  0.64050946,  0.63726823,  0.63526047,
        0.62323271,  0.61965166,  0.61705548,  0.6141438 ,  0.60978056,
```

```python
plt.figure(layout="constrained")
sns.scatterplot(data=df, x="x", y="y")
sns.lineplot(x=df.x, y=p.predict(X = df[["x"]]), color="k")
plt.show()
```



Model coefficients (or other attributes)

The attributes of pipeline steps are not directly accessible, but can be accessed via the `steps` or `named_steps` attributes,

```
p.coef_
```

```
AttributeError: 'Pipeline' object has no attribute 'coef_'
```

```
p.steps
```

```
[('polynomialfeatures', PolynomialFeatures(degree=4)), ('linearregression', LinearRegr
```

```
p.steps[1][1].coef_
```

```
array([  0.        ,   7.39051417, -57.67175293, 102.72227443,
       -55.38181361])
```

```
p.named_steps["linearregression"].intercept_
```

```
np.float64(1.6136636604768198)
```

Other useful bits

```
p.steps[0][1].get_feature_names_out()
```

```
array(['1', 'x', 'x^2', 'x^3', 'x^4'], dtype=object)
```

```
p.steps[1][1].get_params()
```

```
{'copy_X': True, 'fit_intercept': True, 'n_jobs': None, 'positive': False, 'tol': 1e-0
```

Anyone notice a problem?

```
p.steps[1][1].rank_
```

4

```
p.steps[1][1].n_features_in_
```

5

What about step parameters?

By accessing each step we can adjust their parameters (via `set_params()`),

```
p.named_steps["linearregression"].get_params()
```

```
{'copy_X': True, 'fit_intercept': True, 'n_jobs': None, 'positive': False, 'tol': 1e-0
```

```
p.named_steps["linearregression"].set_params
  fit_intercept=False
)
```

▾ LinearRegression

▸ Parameters

```
p.fit(X = df[["x"]], y = df.y)
```

▸ **Pipeline**

▸ PolynomialFeatures

▸ LinearRegression

```
p.named_steps["linearregression"].intercept_
```

```
0.0
```

```
p.named_steps["linearregression"].coef_
```

```
array([  1.61366366,   7.39051417, -57.67175293,
       -55.38181361])
```

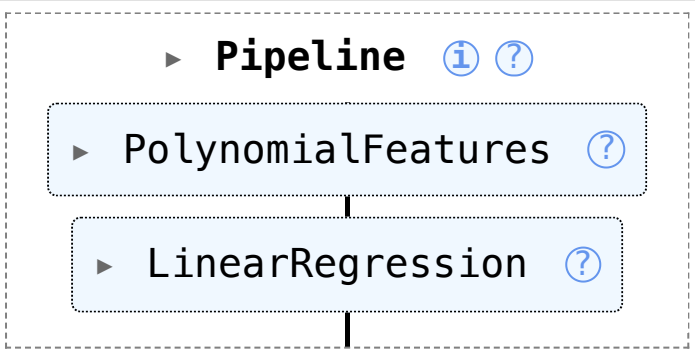
Pipeline parameter names

These parameters can also be directly accessed at the pipeline level, names are constructed as step name + __ + parameter name:

```
p.get_params()
```

```
{'memory': None, 'steps': [('polynomialfeatures', PolynomialFeatures(degree=4)), ('linearregressic
```

```
p.set_params(
  linearregression__fit_intercept=True,
  polynomialfeatures__include_bias=False
)
```



```python
p.fit(X = df[["x"]], y = df.y)
```

```
▸ Pipeline
▸ PolynomialFeatures
▸ LinearRegression
```

```python
p.named_steps["polynomialfeatures"].get_feature_names_out()
```

```
array(['x', 'x^2', 'x^3', 'x^4'], dtype=object)
```

```python
p.named_steps["linearregression"].intercept_
```

```
np.float64(1.6136636604768482)
```

```python
p.named_steps["linearregression"].coef_
```

```
array([  7.39051417, -57.67175293, 102.72227443, -55.38181361])
```

Column Transformers

Column Transformers

Are a tool for selectively applying transformer(s) to column(s) of an array or DataFrame, they function in a way that is similar to a pipeline and similarly have a `make_` helper function.

```python
from sklearn.compose import make_column_transformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
```

```python
ct = make_column_transformer(
  (StandardScaler(), ["volume"]),
  (OneHotEncoder(), ["cover"]),
).fit(
  books
)
ct.get_feature_names_out()
```

```
array(['standardscaler__volume', 'onehotencoder_
       'onehotencoder__cover_pb'], dtype=object)
```

```python
ct.transform(books)
```

```
array([[ 0.12100717,  1.        ,  0.        ],
       [ 0.51996539,  1.        ,  0.        ],
       [ 0.85192299,  1.        ,  0.        ],
       [-1.84637457,  1.        ,  0.        ],
       [-0.43936162,  1.        ,  0.        ],
       [-0.62209057,  1.        ,  0.        ],
       [ 1.1656077 ,  1.        ,  0.        ],
       [-1.31950608,  0.        ,  1.        ],
       [ 0.32809999,  0.        ,  1.        ],
       [ 0.25500841,  0.        ,  1.        ],
       [ 1.9696151 ,  0.        ,  1.        ],
       [-1.2981877 ,  0.        ,  1.        ],
       [ 0.5016925 ,  0.        ,  1.        ],
       [-0.76218277,  0.        ,  1.        ],
       [ 0.57478408,  0.        ,  1.        ]])
```

Keeping or dropping other columns

Another important argument is `remainder` which determines what happens to unspecified columns. The default is `"drop"` which is why `weight` was removed, the alternative is `"passthrough"` which retains untransformed columns.

```
ct = make_column_transformer(
  (StandardScaler(), ["volume"]),
  (OneHotEncoder(), ["cover"]),
  remainder = "passthrough"
).fit(
  books
)
```

```
ct.get_feature_names_out()
```

```
array(['standardscaler__volume', 'onehotencoder_
       'onehotencoder__cover_pb', 'remainder__we
```

```
ct.transform(books)
```

```
array([[ 1.21007174e-01,  1.00000000e+00,  0.00000000e+00,
         8.00000000e+02],
       [ 5.19965391e-01,  1.00000000e+00,  0.00000000e+00,
         9.50000000e+02],
       [ 8.51922992e-01,  1.00000000e+00,  0.00000000e+00,
         1.05000000e+03],
       [-1.84637457e+00,  1.00000000e+00,  0.00000000e+00,
         3.50000000e+02],
       [-4.39361619e-01,  1.00000000e+00,  0.00000000e+00,
         7.50000000e+02],
       [-6.22090574e-01,  1.00000000e+00,  0.00000000e+00,
         6.00000000e+02],
```

```
 [ 1.16560770e+00,  1.00000000e+00,  0.00000000e+00,
   1.07500000e+03],
 [-1.31950608e+00,  0.00000000e+00,  1.00000000e+00,
   2.50000000e+02],
 [ 3.28099989e-01,  0.00000000e+00,  1.00000000e+00,
   7.00000000e+02],
 [ 2.55008407e-01,  0.00000000e+00,  1.00000000e+00,
   6.50000000e+02],
 [ 1.96961510e+00,  0.00000000e+00,  1.00000000e+00,
   9.75000000e+02],
```

Column selection

One lingering issue with the above approach is that we've had to hard code the column names (or use indexes). Often we want to select columns based on their dtype (e.g. categorical vs numerical) this can be done via pandas or sklearn,

```
from sklearn.compose import make_column_selector
```

```
ct1 = make_column_transformer(
  ( StandardScaler(),
    make_column_selector(
      dtype_include=np.number
    )
  ),
  ( OneHotEncoder(),
    make_column_selector(
      dtype_include=[object, bool]
    )
  )
)
```

```
ct2 = make_column_transformer(
  ( StandardScaler(),
    books.select_dtypes(
      include=['number']
    ).columns
  ),
  ( OneHotEncoder(),
    books.select_dtypes(
      include=['object']
    ).columns
  )
)
```

```
<string>:8: Pandas4Warning: For backward compati
See https://pandas.pydata.org/docs/user_guide/mi
```

```python
ct1.fit_transform(books)
```

```
array([[ 0.12100717,  0.35935849,  1.        ,
       [ 0.51996539,  0.93689893,  1.        ,
       [ 0.85192299,  1.32192589,  1.        ,
       [-1.84637457, -1.37326282,  1.        ,
       [-0.43936162,  0.16684502,  1.        ,
       [-0.62209057, -0.41069542,  1.        ,
       [ 1.1656077 ,  1.41818263,  1.        ,
       [-1.31950608, -1.75828978,  0.        ,
       [ 0.32809999, -0.02566846,  0.        ,
       [ 0.25500841, -0.21818194,  0.        ,
       [ 1.9696151 ,  1.03315567,  0.        ,
       [-1.2981877 , -1.37326282,  0.        ,
       [ 0.5016925 ,  0.93689893,  0.        ,
       [-0.76218277, -1.0844926 ,  0.        ,
       [ 0.57478408,  0.07058828,  0.        ,
```

```python
ct2.fit_transform(books)
```

```
array([[ 0.12100717,  0.35935849,  1.        ,
       [ 0.51996539,  0.93689893,  1.        ,
       [ 0.85192299,  1.32192589,  1.        ,
       [-1.84637457, -1.37326282,  1.        ,
       [-0.43936162,  0.16684502,  1.        ,
       [-0.62209057, -0.41069542,  1.        ,
       [ 1.1656077 ,  1.41818263,  1.        ,
       [-1.31950608, -1.75828978,  0.        ,
       [ 0.32809999, -0.02566846,  0.        ,
       [ 0.25500841, -0.21818194,  0.        ,
       [ 1.9696151 ,  1.03315567,  0.        ,
       [-1.2981877 , -1.37326282,  0.        ,
       [ 0.5016925 ,  0.93689893,  0.        ,
       [-0.76218277, -1.0844926 ,  0.        ,
       [ 0.57478408,  0.07058828,  0.        ,
```

```python
ct1.get_feature_names_out()
```

```
array(['standardscaler__volume', 'standardscaler
       'onehotencoder__cover_hb', 'onehotencoder
```

```python
ct2.get_feature_names_out()
```

```
array(['standardscaler__volume', 'standardscaler
       'onehotencoder__cover_hb', 'onehotencoder
```